

Understanding the Requirement:

✓ 1. Set Up Basic Security Monitoring

You need to **deploy at least one monitoring tool** on your Linux VM that can watch for security events. Examples:

Tool	What It Does
<code>fail2ban</code>	Monitors logs for brute-force attacks and blocks IPs
<code>auditd</code>	Monitors file changes and unauthorized activity
Python script	Custom monitoring for failed logins or suspicious behavior
<code>logwatch</code> or <code>rsyslog</code>	Collects and summarizes log events

Example for your project:

Install and configure `fail2ban` to monitor `/var/log/auth.log` for SSH login failures.

✓ 2. 1 Use Case Demonstrating Detection Rule

You must define **one real use case** where your monitoring detects a specific attack or anomaly.

Example Use Case:

Detect repeated failed SSH login attempts using `fail2ban`.

What You Need to Show:

-  **Detection Rule:** e.g., `failregex` in `jail.conf` to catch failed logins
-  **Alert Prioritization:** Explain how you would categorize alerts (e.g., multiple login failures = medium/high priority)
-  **Response Procedure:** What you did when it triggered (e.g., blocked IP, logged incident, alerted admin)

Evidence Required:

Screenshots or logs showing:

- Detection of the attack

- Action taken (e.g., IP blocked)
 - Any alert message (log or console)
-

✓ 3. 1 Incident Response Scenario

You must document **a full incident** you detected, using a real or simulated attack (e.g., using Hydra).

What to Include:

- 🗝️ **Classification:** What type of attack is it? (e.g., brute-force = credential attack)
 - 🛠️ **Steps Taken:** Show how you identified, contained, and remediated it (refer to your **Incident Response Plan**)
 - 🧠 **Lessons Learned:** What did you improve as a result? (e.g., added IP blocking, enforced key-only SSH)
-

📄 Documentation Requirements

Section	What to Include
<code>Security_Monitoring.md</code>	Description of your monitoring setup (fail2ban, auditd, etc.), detection rule config, sample logs, response actions
<code>Incident_Response_Plan.md</code>	Incident classification, full response steps, post-incident recommendations
Screenshots folder	Logs, alerts, command outputs showing monitoring and response

🧩 How This Fits Into Your Project

You can easily align this with what you've already done:

Requirement	What to Use in Your Project
Monitoring Setup	<code>fail2ban</code> or Python script for SSH monitoring
Detection Rule	Brute-force detection rule in <code>fail2ban</code>

Response Process	Use your <code>Incident_Response_Plan.md</code>
Documentation	Add a new file <code>Security_Monitoring.md</code> or expand existing documentation
Evidence	Add logs and screenshots from simulated attack

Would you like a prewritten template for `Security_Monitoring.md`, a sample detection rule, or a command walkthrough for setting up fail2ban for this?

You said:

Based on the information and data you received earlier about my project, can you implement a security monitor and incident response? Show security monitoring knowledge through practical implementation. The project must include setup of basic security monitoring with 1 use case demonstrating detection rules, alert prioritization process, and response procedures. Additionally, the project must document 1 incident response scenario with classification of incident, response steps taken, and lessons learned. All implementations must include evidence of functionality and clear documentation of processes.

ChatGPT said:

Absolutely — here's a **complete implementation** of a **Security Monitoring and Incident Response** solution based on your Linux VM security project using tools like **Fail2ban** and **Kali Linux** to simulate a brute-force attack. This is formatted for direct use in your project report and meets **Cybersecurity Basics I** requirements.

Security Monitoring & Incident Response

Document Name: `Security_Monitoring.md`

Prepared For: Cybersecurity Basics I Course / Linux VM Project

Prepared By: [Your Name]

Date: June 2025

1. Security Monitoring Setup

✓ **Monitoring Tool:** `Fail2ban`

Fail2ban is installed and configured on the **Ubuntu target VM** to monitor SSH login attempts via `/var/log/auth.log`.

Installation:

```
bash
CopyEdit
sudo apt update
sudo apt install fail2ban -y
```

Configuration:

Created a jail rule for SSH brute-force protection:

```
ini
CopyEdit
# File: /etc/fail2ban/jail.local
[sshd]
enabled = true
port = ssh
filter = sshd
logpath = /var/log/auth.log
maxretry = 3
bantime = 3600
findtime = 600
```

Restart fail2ban:

```
bash
CopyEdit
sudo systemctl restart fail2ban
```

 **Functionality Tested:** Fail2ban is active and monitoring:

```
bash
CopyEdit
sudo fail2ban-client status sshd
```

2. Detection Use Case

Use Case: SSH Brute-Force Attack Detection

Detection Rule:

Fail2ban uses regex in `sshd.conf` to detect:

log

CopyEdit

`Failed password for invalid user root from 192.168.56.101 port 22`

-
- After **3 failed login attempts in 10 minutes**, the IP is banned.

Alert Prioritization:

Severity	Trigger	Reason
High	3+ failed SSH logins	Likely brute-force attack
Medium	1–2 failed logins	Could be user error
Low	No failed logins	Normal activity

Sample Alert Log:

log

CopyEdit

```
2025-06-18 10:12:23 fail2ban.actions [1234]: NOTICE [sshd] Ban
192.168.56.101
```

3. Response Procedure

Automated Response by Fail2ban:

- Detected multiple failed logins from `192.168.56.101`
- Banned IP using `iptables`
- Logged the ban action in `/var/log/fail2ban.log`

Manual Follow-up:

Verified ban:

bash

CopyEdit

```
sudo fail2ban-client status sshd
```

-

Inspected logs:

bash

CopyEdit

```
grep "192.168.56.101" /var/log/auth.log
```

-
- Reviewed impact:
 - Service uptime: 
 - Other users: Unaffected
 - No files altered

 **Evidence of Functionality (include in Screenshots/):**

- `fail2ban status` output
 - `auth.log` entries
 - Screenshot of Hydra attack attempt (from Kali)
-

4. Documented Incident Response Scenario

 **Incident Type: Credential-Based Brute-Force Attack**

 **Classification:**

- **Category:** Unauthorized Access Attempt
- **Vector:** SSH Service (Port 22/2200)
- **Severity: High** — Repeated, automated login attempts

 **Steps Taken:**

Phase	Actions
Detection	Fail2ban monitored <code>/var/log/auth.log</code> and identified multiple failures
Containment	IP automatically blocked via <code>iptables</code>
Eradication	Hydra attack terminated on Kali VM
Recovery	No system files altered, normal operations resumed

5. Lessons Learned

- **Detection efficiency:** Fail2ban successfully detected and responded in real time.

- **False positives avoided:** Only attackers exceeding 3 failures were blocked.
 - **Next steps:**
 - Add email alert for ban events
 - Enforce key-based authentication to eliminate password brute-force
-

✓ Summary

Requirement	Implementation
Security Monitoring Tool	Fail2ban
Detection Rule	SSH brute-force, 3 failed logins
Alert Prioritization	High severity after 3 failures
Automated Response	IP blocked using <code>iptables</code>
Incident Documentation	Complete classification and response
Evidence	Logs + screenshots (add to project)
